

Bc. Milan
Janoch

Diplomová práce

Softwarové a informační systémy
2025/2026

Vedoucí práce:
Ing. Martin Dostal, Ph.D.

Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)

Abstrakt

Tato diplomová práce se zabývá využitím velkých jazykových modelů (LLM) pro automatizované end-to-end testování softwaru. Cílem práce je návrh a implementace systému, který na základě uživatelských příběhů generuje robustní automatizované testovací scénáře. Navržené řešení využívá architekturu založenou na autonomním AI agentovi s přístupem k nástrojům pro inspekci prohlížeče v reálném čase. Funkčnost systému byla evaluována pomocí devíti jazykových modelů (z rodin GPT, Claude a Gemini). Samotná evaluace zahrnovala jak generování testů pro plně funkční aplikaci, tak křížovou verifikaci a detekci záměrně zanesených chyb. Výsledky ukazují, že pokročilé modely s podporou reasoningu dokáží efektivně identifikovat defekty a generovat kód s vyšší sémantickou hloubkou než běžné nahrávací nástroje využívané v testování, přičemž dosahují časové úspory ve srovnání s manuálním psaním testů.

Úvod

Automatizované end-to-end testování je klíčové pro kvalitu softwaru, ale jeho údržba a tvorba je časově i finančně nákladná. Cílem této práce je návrh a implementace nástroje, který s využitím LLM autonomně generuje testovací scénáře přímo z uživatelských příběhů.

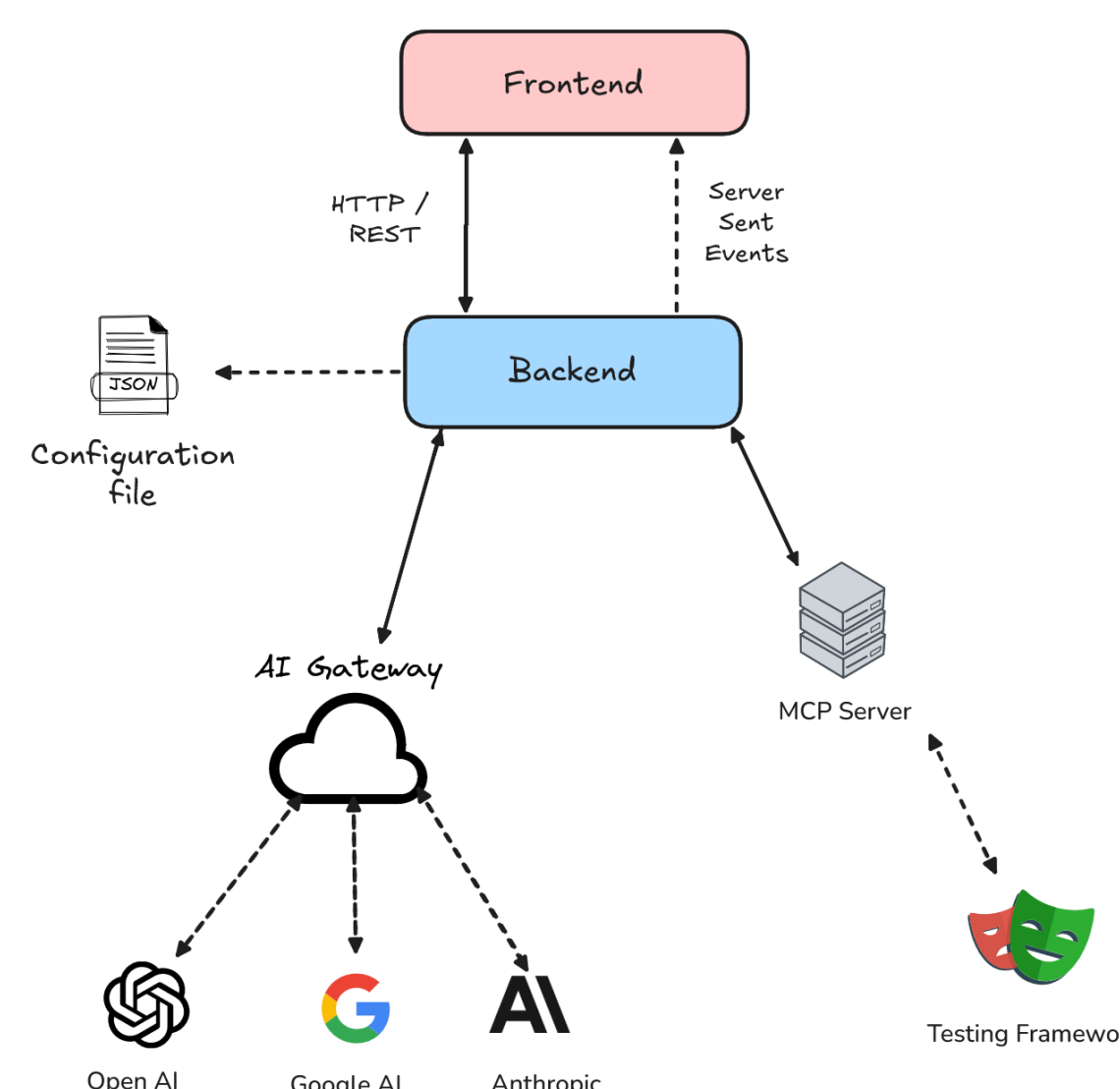
Východiska, analytická část

Tradiční nahrávací přístupy nedokážou zachytit komplexní logiku a vyžadují ruční dopisování asercí. Na druhé straně pokusy o statické generování kódu čistě pomocí jednorázových LLM dotazů často vedou k halucinacím, kdy si modely vymýšlejí neexistující selektory, a k naprosté neschopnosti reagovat na dynamický stav aplikace. Aby mohl nový systém spolehlivě fungovat v reálném vývojovém cyklu, byl proveden rozsáhlý průzkum veřejných instancí nástroje Jira (včetně velkých projektů jako Apache či MongoDB). Tento průzkum odhalil obrovskou nekonzistenci v zápisu uživatelských příběhů, kde často chybí jakákoliv formalizovaná struktura. Tento zjištěný stav si vyžádal návrh vlastního normalizovaného doménového modelu, který slouží jako stabilní a předvídatelná překladová vrstva mezi nestrukturovaným zadáním a LLM.

Hlavní aspekty realizace

Nástroj TestPilotAI byl navržen s ohledem na maximální vývojářský comfort, a proto se instaluje jako lokální NPM balíček. Díky tomuto přístupu se vygenerované testy bezpečně ukládají přímo do vývojového repozitáře, což eliminuje nutnost spoléhat na externí databáze. Samotné jádro systému tvoří autonomní agent, který funguje v iterativní ReAct smyčce (myšlení, akce, pozorování). V této smyčce si agent aplikaci nejprve proaktivně prochází, zkoumá, strukturálně ověřuje její funkčnost a až na základě těchto faktů píše finální testovací kód. Pro fyzické ovládání prohlížeče je využit Playwright MCP běžící jako nezávislý

a bezpečně izolovaný proces, se kterým agent komunikuje přes standardizované rozhraní pomocí standardního vstupu a výstupu. Systém navíc efektivně sjednocuje komunikaci s LLM pomocí vrstvy Vercel AI SDK (AI Gateway), což umožňuje okamžité přepínání mezi devíti modely (OpenAI, Anthropic, Google AI) bez nutnosti měnit vnitřní logiku. Spolehlivost výstupu nakonec zajišťuje validační pipeline, ve které vygenerovaný kód prochází in-memory kompilací. Pokud dojde k detekci sémantické nebo syntaktické chyby, systém ji automaticky pošle modelu zpět k opravě v rámci dedikované samoopravné smyčky.



Koncepční architektura systému

Dosažené výsledky

- Detekce skrytých defektů:** Evaluace proběhla na 8 scénářích vůči aplikaci s 11 záměrně zanesenými chybami, jako byla chybějící Toast notifikace či únik administrátorského menu běžnému uživateli. Pokročilejší modely s reasoningem (GPT-5, Claude Sonnet 4.5, Gemini 2.5 Pro) úspěšně odhalily všech 11 chyb.
- Oprava testů:** Díky samoopravné smyčce a zpátné vazbě z chybových report dokázal systém opravit 19 původně selhávajících testů.
- Časová a ekonomická úspora:** Vygenerování E2E scénáře trvá agentovi 2-5 minut, zatímco manuální nahrávání zabere průměrně 15 minut. Zároveň se ukázalo, že menší modely dokáží generovat kód v řádu desítek centů dolarů.

Závěr

Prototyp prokazuje, že integrace LLM se standardem MCP mění dosavadní paradigma automatizovaného testování z pouhého statického generování na dynamickou interakci. Nástroj spolehlivě eliminuje limity běžných nahrávacích nástrojů a šetří čas nutný pro vývoj. Díky schopnosti křížové verifikace s akceptačními kritérii funguje jako kontrolní mechanismus, který odhaluje defekty ještě před samotným spuštěním výsledného testu, což z něj činí efektivního asistenta pro agilní vývojové týmy.